MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 1
VERİ VE VERİ YAPILARI

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Dersin Amacı

- Mevcut veri yapıları ve veri modellerini anlamak
- Algoritmaları incelemek, mevcut çözümleri anlamak
- Bilgisayar ve yazılım dünyasına ait bazı kavramlar, terimler ve sözcükleri öğrenmek
- Bağlı liste, ağaç, graf gibi çeşitli veri modellerini kavramak
- Programların bellek gereksinimi ve çalışma hızlarının hesabını yapmak
- Arama ve sıralama algoritmalarını anlamak.

Ders Konuları

- Hafta 1: Veri ve Veri Yapıları
- Hafta 2: Algoritma Analizi
- Hafta 3: Dizi ve Koleksiyonlar
- Hafta 4: Bağlı Listeler
- Hafta 5: Yığın
- Hafta 6: Kuyruk
- Hafta 7: Öncelikli Kuyruk ve Yığınlar
- Hafta 8: Ağaçlar
- Hafta 9: Ayrık Kümeler
- Hafta 10: Çizgeler
- Hafta 11: Çizge Tabanlı Algoritmalar
- Hafta 12: Sıralama Algoritmaları
- Hafta 13: Arama Algoritmaları
- Hafta 14: Özel Veri Tipleri

Ders Kaynakları

- Data Structures and Algorithms in Java Michael T. Goodrich and Roberto Tamassia
- Algoritmalara Giriş, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest ve Clifford Stein
- Algoritmalar Teoriden Uygulamalara Vasif Vagifoğlu Nabiyev

Ders Değerlendirme Ölçütleri

ARASINAV (VİZE)	%40
DÖNEM SONU SINAVI (FİNAL)	%60
DEVAM (%70)	Geçme-Kalma

Ders Araçları

- Programlama Dili: C
  - C Programlama Dili genel amaçlı orta seviyeli ve yapısal bir programlama dilidir.

- IDE: Dev C++
  - Dev C++, C ve C++ dillerini destekleyen, GNU GPL lisanslı ile dağıtılan özgür bir tümleşik geliştirme ortamıdır.

Ders İçeriği: Hafta 1

- Temel Kavramlar
- Veri Kavramı
- Veri Tipleri
- Veri Yapıları
- Veri Modeli

Temel Kavramlar

Bellek (Memory)

- Herhangi bir elektronik cihazda çeşitli programların çalıştırıldığı, geçici veya kalıcı bilgilerin bulunacağı hafıza alanlarıdır.
- ROM (Read-Only Memory)
  - Üzerindeki bilgiler kalıcıdır.
  - PROM: Programlanabilir ROM
  - EPROM: Silinebilir programlanabilir ROM
  - EEPROM: Elektrikle silinebilir programlanabilir ROM
  - Flash Bellek: Normalde salt okunur olan ancak silinebilen bir tür EEPROM.
    - Önemli özelliği ise içeriğini herhangi bir güç olmadan tutabilir. Elektrik gitse de veriler korunur.

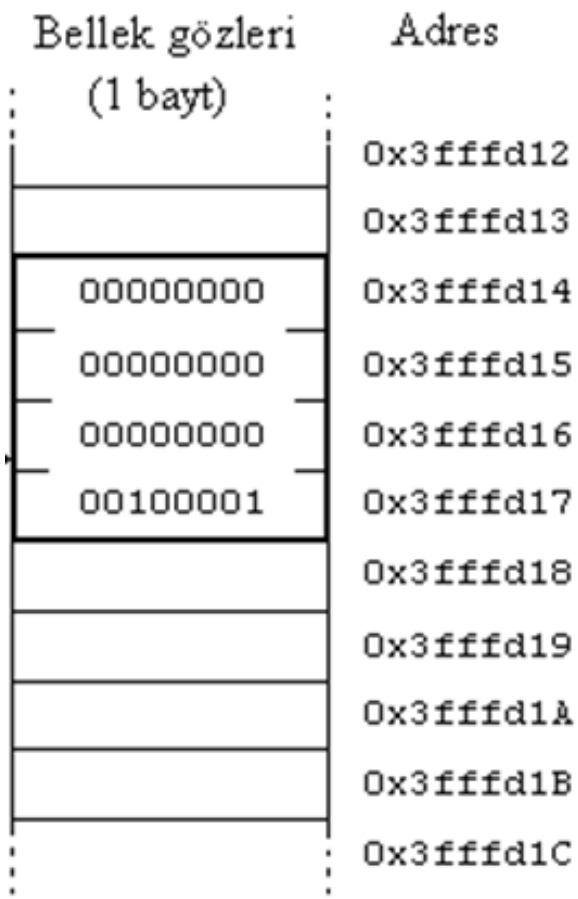
Temel Kavramlar

- Bellek (Memory)
  - RAM (Random Access Memory)
    - Üzerindeki bilgiler okunabilir-yazılabilirdir ve elektrik kesildiğinde bilgiler silinir.
      - DRAM: Dinamik, ucuz, ana bellek olarak kullanılır.
      - SRAM: Statik, pahalı, daha hızlı, önbellek olarak kullanılır.
      - SDRAM: (Synshronous) Saat işareti eklenmiştir ve eş zamanlı bir arayüze sahiptir. Gelen komutları ardışık düzene sokan dahili bir sonlu durum makinesini kullanır.
      - DDR SDRAM: (Double Date Rate) Saat işaretinin hem çıkan hem de inen kenarlarında erişim yapılır. DDR, DDR2, DDR3, DDR4 ve DDR5 çeşitleri vardır.

Temel Kavramlar

➢ Bellek (Memory)

- RAM (Random Access Memory)
  - Sıralı kaydetme gözlerinden oluşmuştur.
  - Her göze bir adres atanmıştır.
  - Bu adreslerin değerleri 0 ile belleğin sahip olduğu üst değere bağlı olarak değişebilir.
  - ÖR:
    - 1GB RAM
    - $1024^3 = 1073741824$ adet gözden oluşur.



Veri Kavramı

Veri Nedir?

- Simge, sayı, harf, karakter, resim, ses, video gibi saf ve işlenmemiş gerçekler topluluğu olarak tanımlanabilir.

Bilgisayarda Veri Kavramı

- Veriler bilgisayarda ise ham olarak 1 ve 0'lardan oluşan bit (binary digit) dizisi olarak tutulur.
- Yani işlemcinin anlayıp işleyebileceği şekildedir.

Veri Kavramı

Programlamada Veri Kavramı

- Programlama dillerinde veriler değişken şeklinde tanımlanır ve kullanılır.
- Bir değişken tanımlandığında ve değer ataması yapıldığında o değişkene (veriye) dört temel özellik eşlik eder:
  - Değişkenin adı,
  - Değişkenin tipi,
  - Değişkenin değeri,
  - Değişkenin bellekteki adresi.

Veri Kavramı

- Verinin Bellekte Tutulması
  - 1 ve 0'lardan oluşan bit dizisi şeklindedir.
  - Örneğin 62 sayısı bellekte üç farklı yöntemle tutulabilir:
    - Sayının ikili tabandaki karşılığı
      - 111110
    - Her bir karaktere özel kod verilmiş olan ASCII kodlarına göre
      - 00110110 00110010
    - BCD kodlama
      - 0110 0010

Veri Kavramı

➢ Verinin Bellekte Tutulması

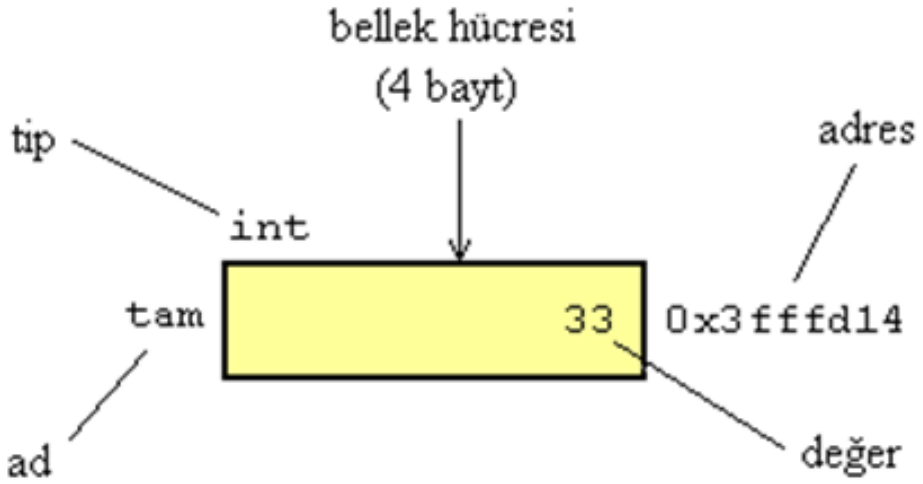
- Veri tipine göre bellekte kaplanacak alan ayrılır:

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615
float	4 byte	1.2E-38 to 3.4E+38
double	8 byte	2.3E-308 to 1.7E+308
long double	10 byte	3.4E-4932 to 1.1E+4932

Veri Kavramı

➢ Verinin Bellekte Tutulması

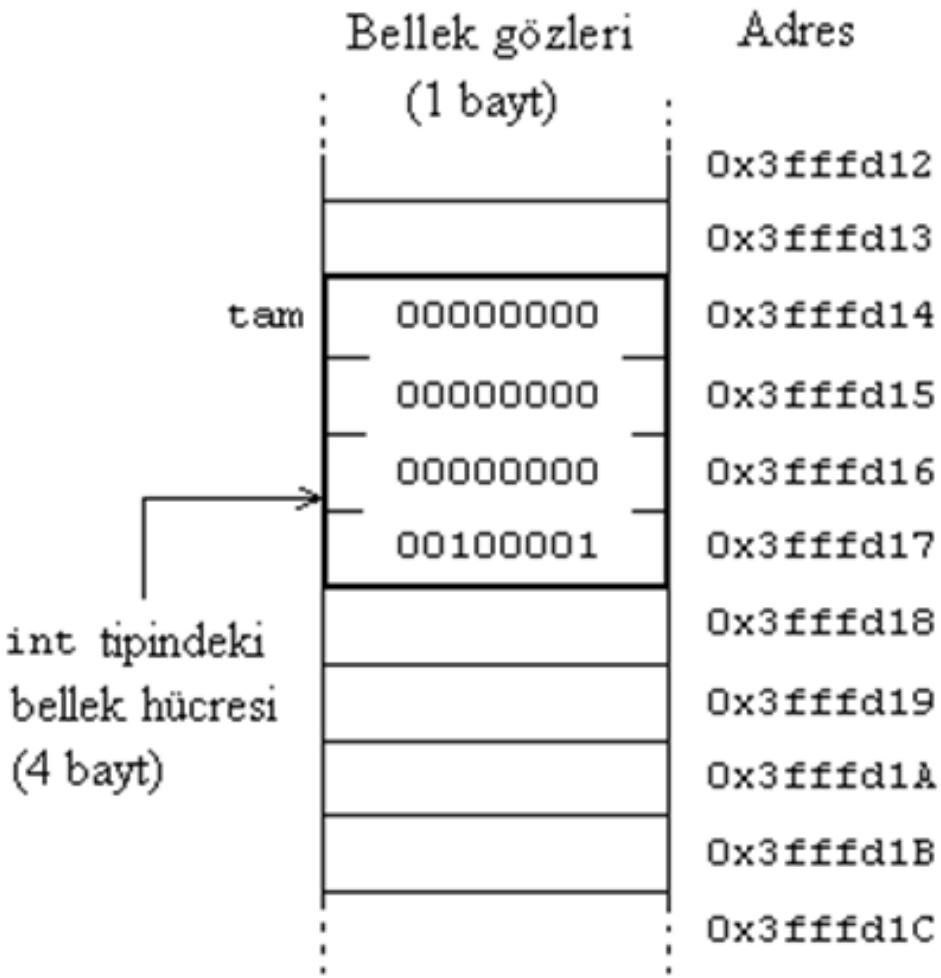
- Örnek:
  - int tam=33; değişkeni için bellekte 4 bayt büyüklüğünde bir hücre ayrılır. O hücreye 33 sayısının ikilik karşılığı 4 baytlık (32 bitlik) olarak yazılır: 00000000 00000000 00000000 00100001
  - Bellek adresleri genellikle onaltılık sayı sisteminde ifade edilir. Değişkeninin adresi 0x3FFFD14 olsun.



Veri Kavramı

➢ Verinin Bellekte Tutulması

- Örnek:

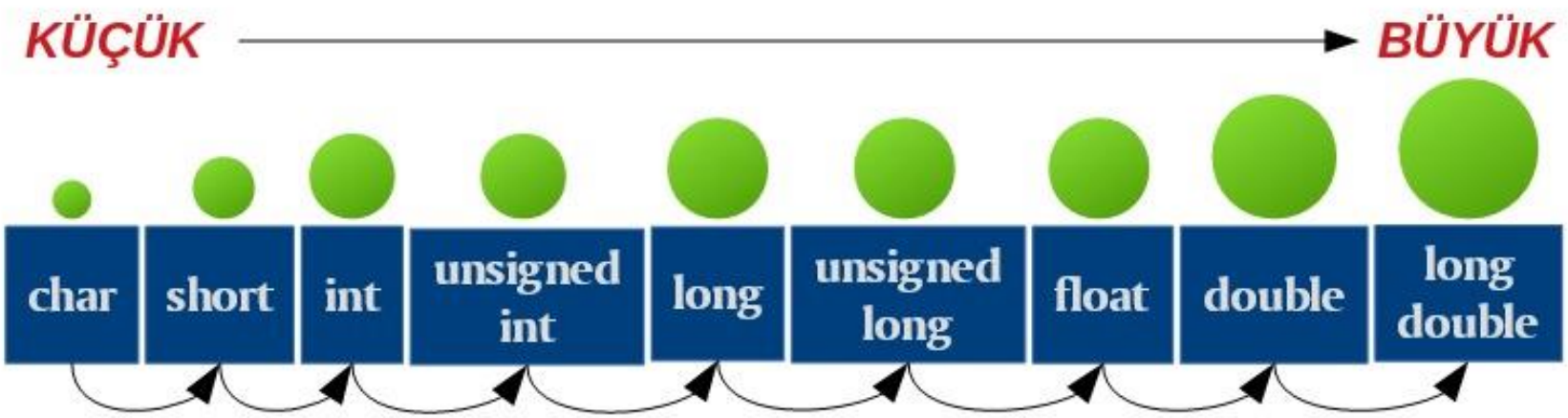


Veri Tipleri

➢ Neden Veri Tipi

- Farklı veri tiplerinin büyüklükleri yani bellekte kaplayacakları bit sayısı ve onlarla yapılabilecek işlemler birbirlerinden farklıdır.
- Örneğin, sayılarla dört işlem yapabiliriz, ama metinlerle yapamayız.
- Bu nedenle, bazı diller verileri tiplere ayırır. (Örneğin C). Değişken tanımlarken onun hangi tip veriyi tutacağını belirtir.
- Böylece, ana bellekte o değişkene yetecek bir yer ayırır ve o veri tipine uygun işlemlerin yapılmasına izin verir.

C Veri Tipleri



Veri Yapıları

➢ Veri Yapısı (Data Structure)

- Verilerin bellekte tutulma şeklini veya düzenini gösterir.
- Tüm programlama dillerinin, genel olarak, tamsayı, kesirli sayı, karakter ve sözcük saklanması için temel veri yapıları vardır.
- Bir program değişkeni bile basit bir veri yapısı olarak kabul edilebilir.
- Programcı bu veri yapılarını, bunların bellekte nasıl saklandığı konusuyla ilgilenmeksizin bolca kullanır.
- Ayrıca, programcıların temel veri yapıları dışında yeni veri yapıları tanımlayabilmesi için programlama dillerinde bir çok özellik vardır.

Veri Yapıları

➢ Neden Veri Yapılarına İhtiyaç Var?

- Bilgisayar yazılımları gün geçtikçe daha karmaşık bir hal almaktadır.
  - Örneğin 8 milyar sayfanın indekslenmesi (Google)
- Yazılımların programlanması ve yönetimi zorlaşmaktadır.
- Temiz kavramsal yapılar ve bu yapıları sunan çerçeve programları, daha etkin ve daha doğru program yazmayı sağlar.
- Verimli Veri Yapıları, Verimli Algoritmalar!

Veri Yapıları

➢ Neden Veri Yapılarına İhtiyaç Var?

- Her bir satır başına ortalama 10 kelime ve yine yaklaşık 20 satırdan oluşan 3000 metin koleksiyonu düşünelim.
  - 600.000 Kelime
- Bu metinler içinde «dünya» kelimesinin tüm eşleşmelerini bulmak isteyelim.
- Doğru eşleştirme için yapılacak karşılaştırmanın 1 sn. sürdüğünü varsayalım.
- Çözüm 1: Sıralı eşleştirme: 1 sn. x 600.000 kelime= 166 saat
- Çözüm 2: İkili Arama: Kelimeler sıralanır, sadece tek yarıda arama yapılır ve toplam adım sayısı= 20 saniye.
- 20 saniye veya 166 saat!

Veri Yapıları

➢ Programlamada Veri Yapılarının Sınıflandırılması

- Temel Veri Yapıları
  - Daha çok programlama dilleri tarafından doğrudan değişken veya sabit bildirimi yapılarak kullanılır.
  - short, int , double , char , decimal, object, string vb.
- Tanımlamalı (Birleşik) Veri Yapıları
  - Kendisinden önceki tanımlamalı veya temel veri yapıları üzerine kurulurlar; yani, önceden geçerli olan veri yapıları kullanılarak sonradan tanımlanırlar.
  - struct, class, interface vb.
- Veri yapıları programlama dilinin özelliklerine bağlıdır.
- Hem temel hem de tanımlamalı veri yapıları değer veya başvuru türünde olabilir.

Veri Yapıları

➢ struct Veri Yapısı

- Farklı veri türlerini tek bir yapı altında toplamaya ihtiyaç duyulduğunda struct kullanılabilir.
- Temel amaç verinin bütününü temsil edecek alt veri türlerini bir yapı altında toplamaktır.
- Verinin organize edilmesi için kullanılan en eski programlama bileşenlerinden biridir.
- struct yapısı değer (value) tiplidir ve kullanımı tamamlandıktan sonra bellekten hızlıca kaldırılabilir.
- Bu açıdan bakıldığında sınıf yapısına kıyasla performansı daha iyidir.
- struct yapısı kalıtımın uygulanmasını desteklemez ancak arayüz kalıtımını destekler

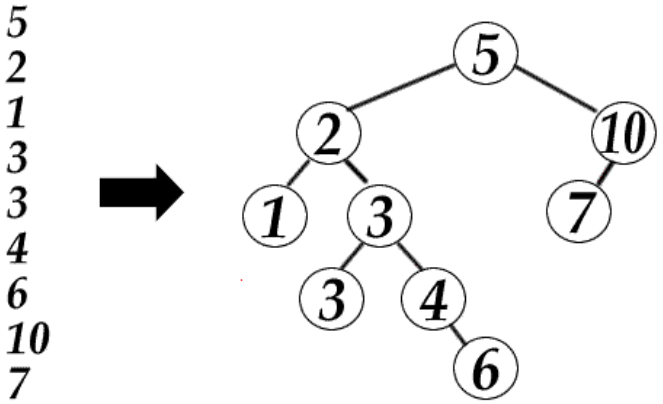
Veri Yapıları

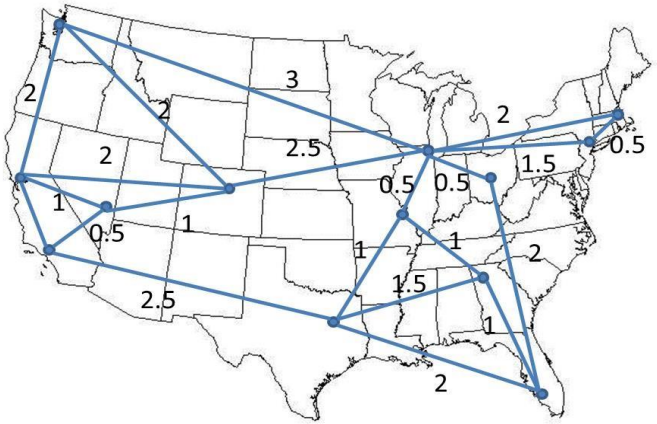
➢ class Veri Yapısı

- Nesne yönelimli programlamanın temel öğesidir.
- İçerisinde field, property, method, operatör gibi pek çok farklı üye barındırır.
- Referans tipli bir veri türüdür.
- Class(Sınıf) ve interface(arayüz) kalıtımını destekler.

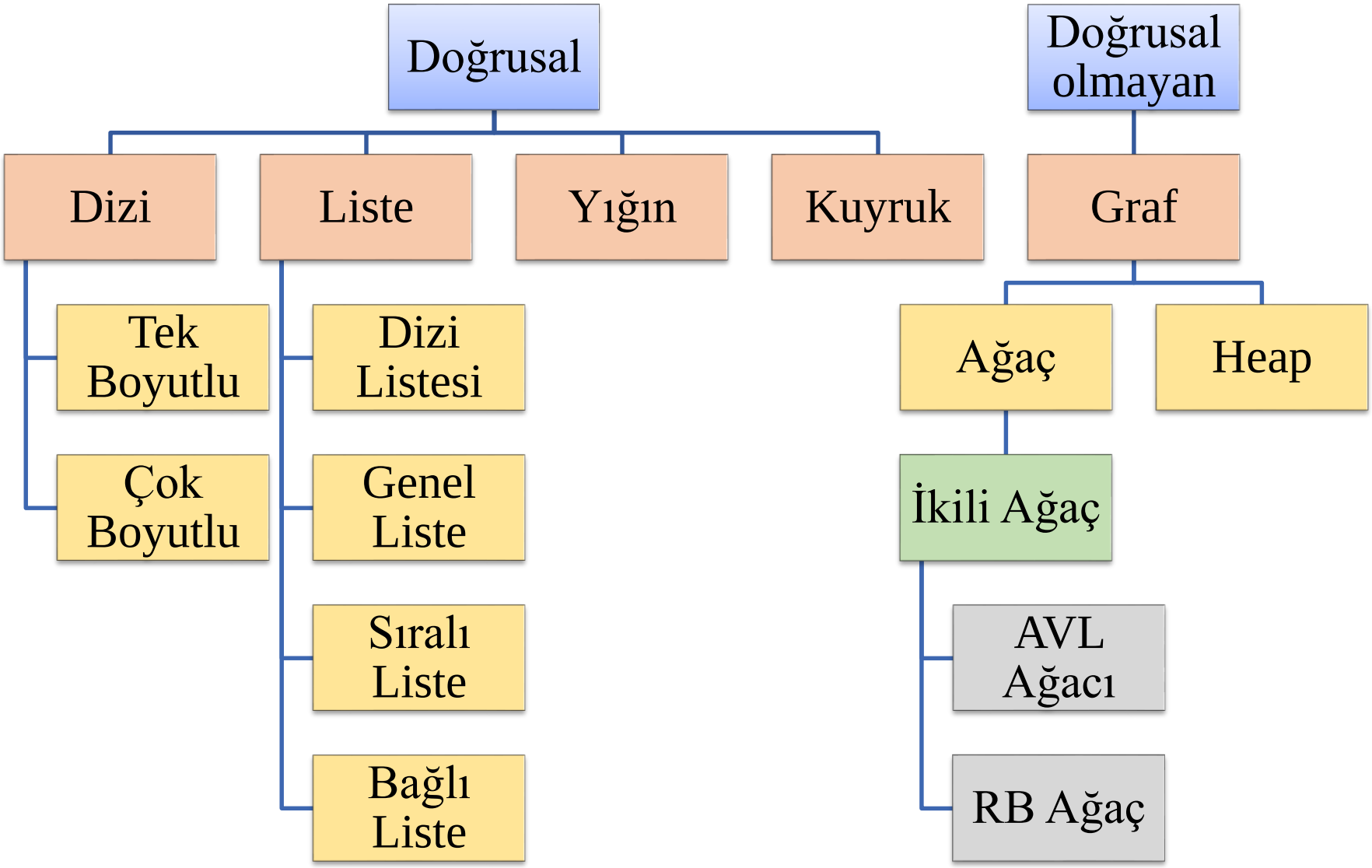
Veri Modeli

- Veri modeli (Data Model), verilerin birbirleriyle ilişkisel veya sırasal durumunu gösterir.
- Bilgisayar ortamında uygulanacak tüm matematik ve mühendislik problemleri bir veri modeline yaklaştırılarak veya yeni veri modelleri tanımlaması yapılarak çözülebilmektedir.
  - Veriniz üzerinde hızlı arama yapmak istiyorsanız bir ağaç modeli kullanabilirsiniz.
  - Şehirler arasındaki mesafeleri hesaplayan bir yazılım geliştireceğiniz durumda ise graf modelini kullanabilirsiniz.





Veri Modeli


Doğrusal
Doğrusal olmayan
Dizi
Liste
Yığın
Kuyruk
Graf
Tek Boyutlu
Çok Boyutlu
Dizi Listesi
Genel Liste
Sıralı Liste
Bağlı Liste
Ağaç
Heap
İkili Ağaç
AVL Ağacı
RB Ağaç

Veri Modeli

Veri Modeli	Artıları	Eksileri
Dizi	Hızlı ekleme ve çok hızlı erişim(indis biliniyorsa).	Yavaş arama, yavaş silme ve sabit boyut.
Sıralı Dizi	Sıralanmamış diziye göre daha hızlı arama.	Yavaş arama, yavaş silme ve sabit boyut.
Yığın	Son giren, ilk çıkar(last-in, first-out) erişimi sağlar.	Diğer öğelere yavaş erişim.
Kuyruk	İlk giren, ilk çıkar(first-in, first-out) erişimi sağlar.	Diğer öğelere yavaş erişim.
Bağlı Liste	Hızlı ekleme ve silme.	Yavaş arama.
Heap	Hızlı ekleme ve silme.	Diğer öğelere yavaş erişim. Başta en büyük öğeye erişim.
İkili Ağaç	Hızlı arama, ekleme ve silme(ağaç dengeli kalmışsa).	Silme algoritması karmaşık.
Graf	Gerçek-dünya problemlerini modelleyebilme.	Bazı algoritmaları yavaş çalışmakta ve karmaşıklığı yüksek.